ORF307
Interior Point Methods I

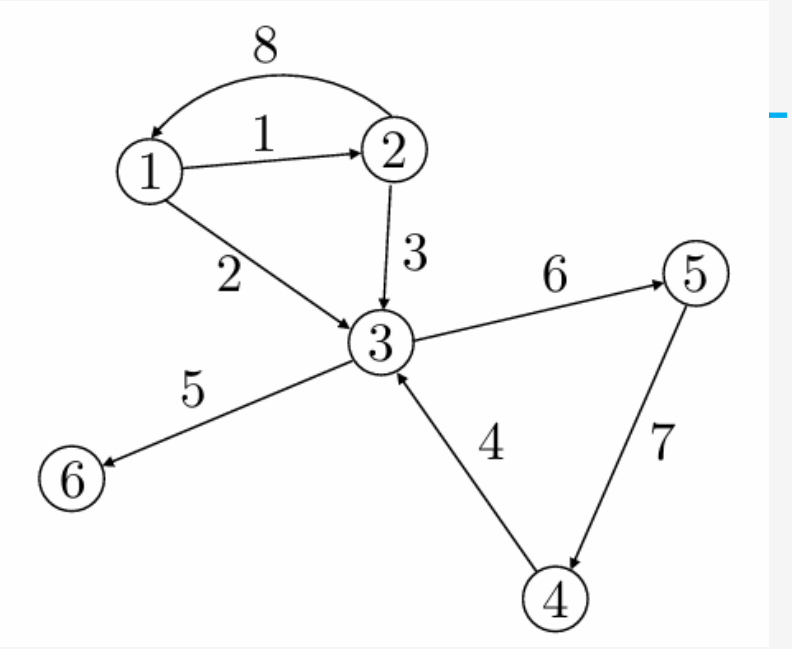
Ioannis Akrotirianakis

Today's topics

- History
- Newton's method
- Central path
- Primal-dual path-following algorithm
- Logarithmic barrier functions

- **Recap**

Arc-node incidence matrix



Definition: a matrix $A \in R^{m \times n}$ is called the incidence matrix of a network if

$$A_{ij} = \begin{cases} 1, & \text{If arc } j \text{ starts at node } i \\ -1, & \text{If arc } j \text{ ends at node } i \\ 0, & \text{otherwise} \end{cases}$$

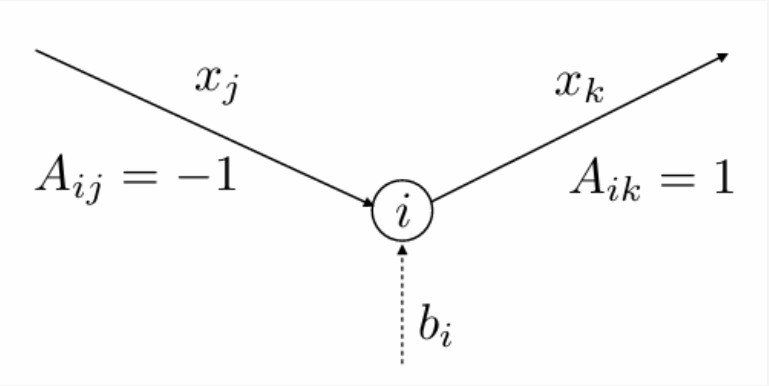
Note: each column has one -1 and one 1

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & -1 & -1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \end{bmatrix}$$

External supply

Supply vector $b \in R^m$:

- b_i is the external supply at node i
  - If $b_i < 0$, it represents demand at node i
- We must have $1^T b = 0$
  - Total supply=Total demand



Balance equations

$$\sum_{j=1}^{n} A_{ij} x_j = (Ax)_i = b_i, \ \forall i \quad \Rightarrow \quad Ax = b$$

Total outgoing Flow at node i

Supply at node i

Minimum cost network flow problem

$$\min c^T x \qquad c, x, u \in R^n$$

$$\text{s.t. } Ax = b \qquad b \in R^m$$

$$0 \leq x \leq u \qquad A \in R^{m \times n}$$

- c_i is the unit cost of flow through arc i
- x_i must be nonnegative
- u_i is the maximum flow capacity of arc i
- Many network optimization problems are a special case of this model

Integrality theorem

Theorem: If a polyhedron $P = \{x \in R^n \mid Ax = b,\ x \geq 0\}$, where $A \in R^{m \times n}$ is a totally unimodular matrix and $b \in R^m$ is an integer vector, then all extreme points of P are integer vectors.

Proof: recall that all extreme points are basic feasible solutions with $x_B = A_B^{-1} b$ and $x_i = 0, \forall i \notin B$.

Also, A_B^{-1} has integer elements because A is totally unimodular matrix

Note also that b has integer elements

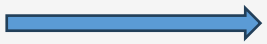
Hence the vector $\mathrm{x_B} = A_B^{-1} b$ has integer elements, and so does x.

■

Implications for network and combinatorial optimization

Minimum cost network flow

$$\min\ c^T x$$
$$s.t.\ \ Ax = b$$
$$0 \le x \le u$$



If b and u are integral vectors, then the optimal solution x^* is integral too

Integer Linear Program (ILP)

$$\min\ c^T x$$
$$s.t.\ \ Ax = b$$
$$0 \le x \le u$$
$$x \in Z^n$$

ILPs are very difficult to solve (NP-complete)

If A is totally unimodular and b, u are integral, then we can relax the integrality constraint $x \in Z^n$ and solve the LP instead.

- **History**

A brief history of LP algorithms – IPM focus

1940s:

- Foundations of economics and logistics (Koopmans, Kantorovich)
- **1947**: development of Simplex method (Dantzig)

1950s-70s:

- Applications expand to engineering, OR, Computer Science, …
- **1979**: development of Ellipsoid method – 1st polynomial algorithm for LP (Khachiyan)

1980s:

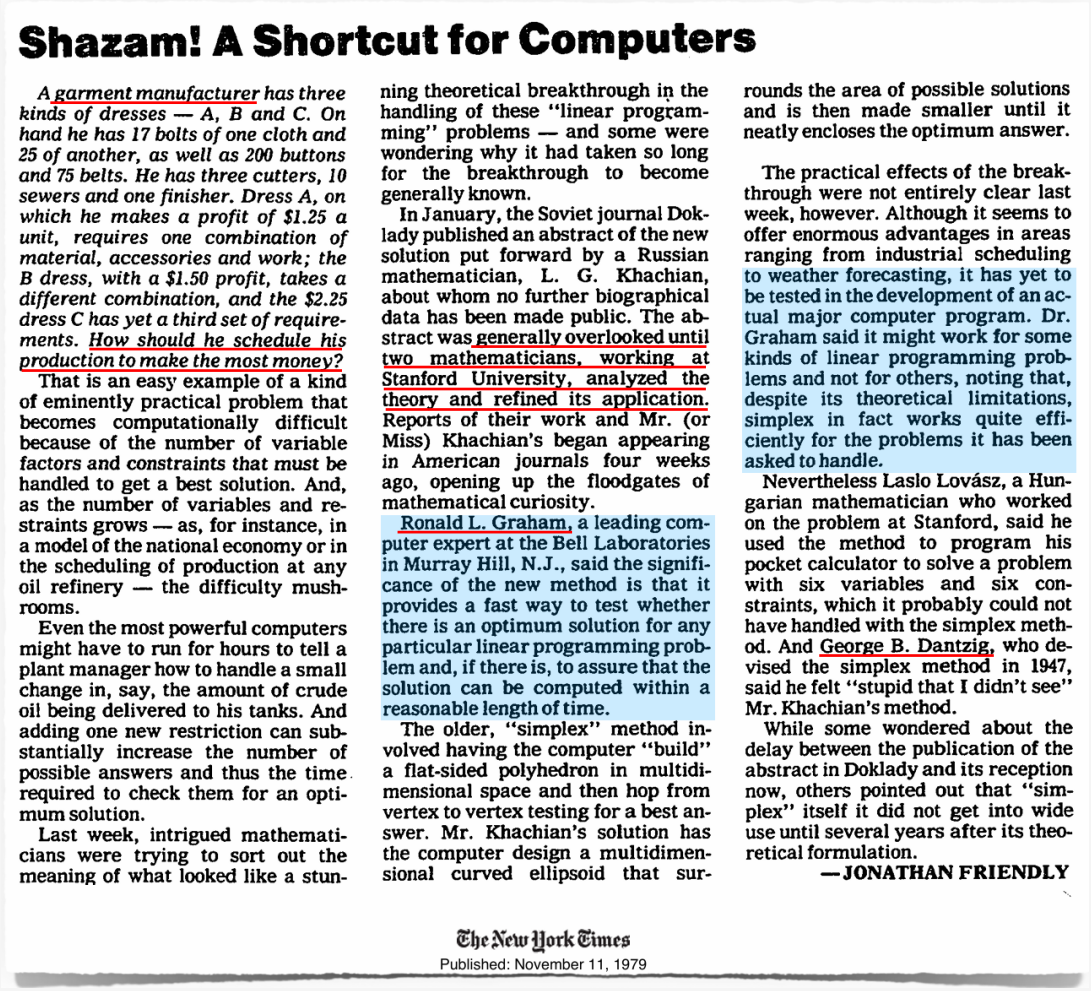
- Development of polynomial time algorithms for LP
- **1984**: development of Interior Point Method- practical polynomial time algorithms (Karmarkar)

Present times:

- Continued algorithm development. Expansion to very large-scale problems

Ellipsoid method

- **Developed by Leonid Khachian (Russia 1979)**
- **1st polynomial time algorithm for LP**
  - Note: worst-case polynomial time
- **Practical performance:**
  - Most of the times much slower than Simplex
  - Notoriously inefficient algorithm – rarely used
- **Benefits:**
  - Motivated new research directions
  - IPM is a result of this effort



Shazam! A Shortcut for Computers

A garment manufacturer has three kinds of dresses — A, B and C. On hand he has 17 bolts of one cloth and 25 of another, as well as 200 buttons and 75 belts. He has three cutters, 10 sewers and one finisher. Dress A, on which he makes a profit of $1.25 a unit, requires one combination of material, accessories and work; the B dress, with a $1.50 profit, takes a different combination, and the $2.25 dress C has yet a third set of requirements. How should he schedule his production to make the most money?

That is an easy example of a kind of eminently practical problem that becomes computationally difficult because of the number of variable factors and constraints that must be handled to get a best solution. And, as the number of variables and restraints grows — as, for instance, in a model of the national economy or in the scheduling of production at any oil refinery — the difficulty mushrooms.

Even the most powerful computers might have to run for hours to tell a plant manager how to handle a small change in, say, the amount of crude oil being delivered to his tanks. And adding one new restriction can substantially increase the number of possible answers and thus the time required to check them for an optimum solution.

Last week, intrigued mathematicians were trying to sort out the meaning of what looked like a stunning theoretical breakthrough in the handling of these "linear programming" problems — and some were wondering why it had taken so long for the breakthrough to become generally known.

In January, the Soviet journal Doklady published an abstract of the new solution put forward by a Russian mathematician, L. G. Khachian, about whom no further biographical data has been made public. The abstract was generally overlooked until two mathematicians, working at Stanford University, analyzed the theory and refined its application. Reports of their work and Mr. (or Miss) Khachian's began appearing in American journals four weeks ago, opening up the floodgates of mathematical curiosity.

Ronald L. Graham, a leading computer expert at the Bell Laboratories in Murray Hill, N.J., said the significance of the new method is that it provides a fast way to test whether there is an optimum solution for any particular linear programming problem and, if there is, to assure that the solution can be computed within a reasonable length of time.

The older, "simplex" method involved having the computer "build" a flat-sided polyhedron in multidimensional space and then hop from vertex to vertex testing for a best answer. Mr. Khachian's solution has the computer design a multidimensional curved ellipsoid that surrounds the area of possible solutions and is then made smaller until it neatly encloses the optimum answer.

The practical effects of the breakthrough were not entirely clear last week, however. Although it seems to offer enormous advantages in areas ranging from industrial scheduling to weather forecasting, it has yet to be tested in the development of an actual major computer program. Dr. Graham said it might work for some kinds of linear programming problems and not for others, noting that, despite its theoretical limitations, simplex in fact works quite efficiently for the problems it has been asked to handle.

Nevertheless Laslo Lovász, a Hungarian mathematician who worked on the problem at Stanford, said he used the method to program his pocket calculator to solve a problem with six variables and six constraints, which it probably could not have handled with the simplex method. And George B. Dantzig, who devised the simplex method in 1947, said he felt "stupid that I didn't see" Mr. Khachian's method.

While some wondered about the delay between the publication of the abstract in Doklady and its reception now, others pointed out that "simplex" itself it did not get into wide use until several years after its theoretical formulation.

— JONATHAN FRIENDLY

The New York Times
Published: November 11, 1979

Interior Point Method (IPM)

- Main optimization algorithm in the 1980s-1990s
- **Developed by Nerendra Karmarkar (Bell Labs-1984)**
- **Competitive with Simplex**
  - Often faster than Simplex in large LPs
- Extended to solve **Nonlinear Optimization problems**
  - Practically efficient algorithms for Convex and Nonconvex Optimization problems

The New York Times

NEW YORK, MONDAY, NOVEMBER 19, 1984

AMERICANS IN POLL VIEW GOVERNMENT MORE CONFIDENTLY

But Postelection Inquiry Also Finds Most Think Reagan Will Ask Rise in Taxes

TREASURY MAY ASK INCREASES IN TAXES OF SOME CONCERNS

OTHERS COULD PAY LESS

Albany Leaders Predicting a Cut In Income Taxes

Cocaine Traffickers Kill 17 in Peru Raid On Antidrug Team

Vote Comes to a 'Homeland,' But African Problems Linger

Breakthrough in Problem Solving

Homeless Spend Nights in City Welfare Office

Breakthrough in Problem Solving

By JAMES GLEICK

A 28-year-old mathematician at A.T.&T. Bell Laboratories has made a startling theoretical breakthrough in the solving of systems of equations that often grow too vast and complex for the most powerful computers.

The discovery, which is to be formally published next month, is already circulating rapidly through the mathematical world. It has also set off a deluge of inquiries from brokerage houses, oil companies and airlines, industries with millions of dollars at stake in problems known as linear programming.

These problems are fiendishly complicated systems, often with thousands of variables. They arise in a variety of commercial and government applications, ranging from allocating time on a communications satellite to routing millions of telephone calls over long distances. Linear programming is particularly useful whenever a limited, expensive resource must be spread most efficiently among competing users. And investment companies use the approach in creating portfolios with the best mix of stocks and bonds.

Faster Solutions Seen

The Bell Labs mathematician, Dr. Narendra Karmarkar, has devised a radically new procedure that may speed the routine handling of such problems by businesses and Government agencies and also make it possible to tackle problems that are now far out of reach.

"This is a path-breaking result," said Dr. Ronald L. Graham, director of mathematical sciences for Bell Labs in Murray Hill, N.J. "Science has its moments of great progress, and this may well be one of them."

Because problems in linear programming can have billions or more possible answers, even high-speed computers cannot check every one. So computers must use a special procedure, an algorithm, to examine as few answers as possible before finding the best one — typically the one that minimizes cost or maximizes efficiency.

A procedure devised in 1947, the simplex method, is now used for such prob-

Continued on Page A19, Column 1

Newton’s method

Newton's root finding method (univariate functions)

- **Goal**: solve the univariate equation (find the roots of the univariate function $h: R \to R$):

$$h(x) = 0$$

- The method consists of **3 main iterations** (we start with **initial point x^0** at iteration 0)

1. Approximate h by its **1st order Taylor expansion** at the current point x^k

$$h(x) \approx \hat{h}(x) = h(x^k) + \frac{dh(x^k)}{dx}(x - x^k)$$

2. Solve the **linear equation**:

$$h(x^k) + \frac{dh(x^k)}{dx}(x - x^k) = 0 \quad \to \quad \hat{x}: \ solution$$

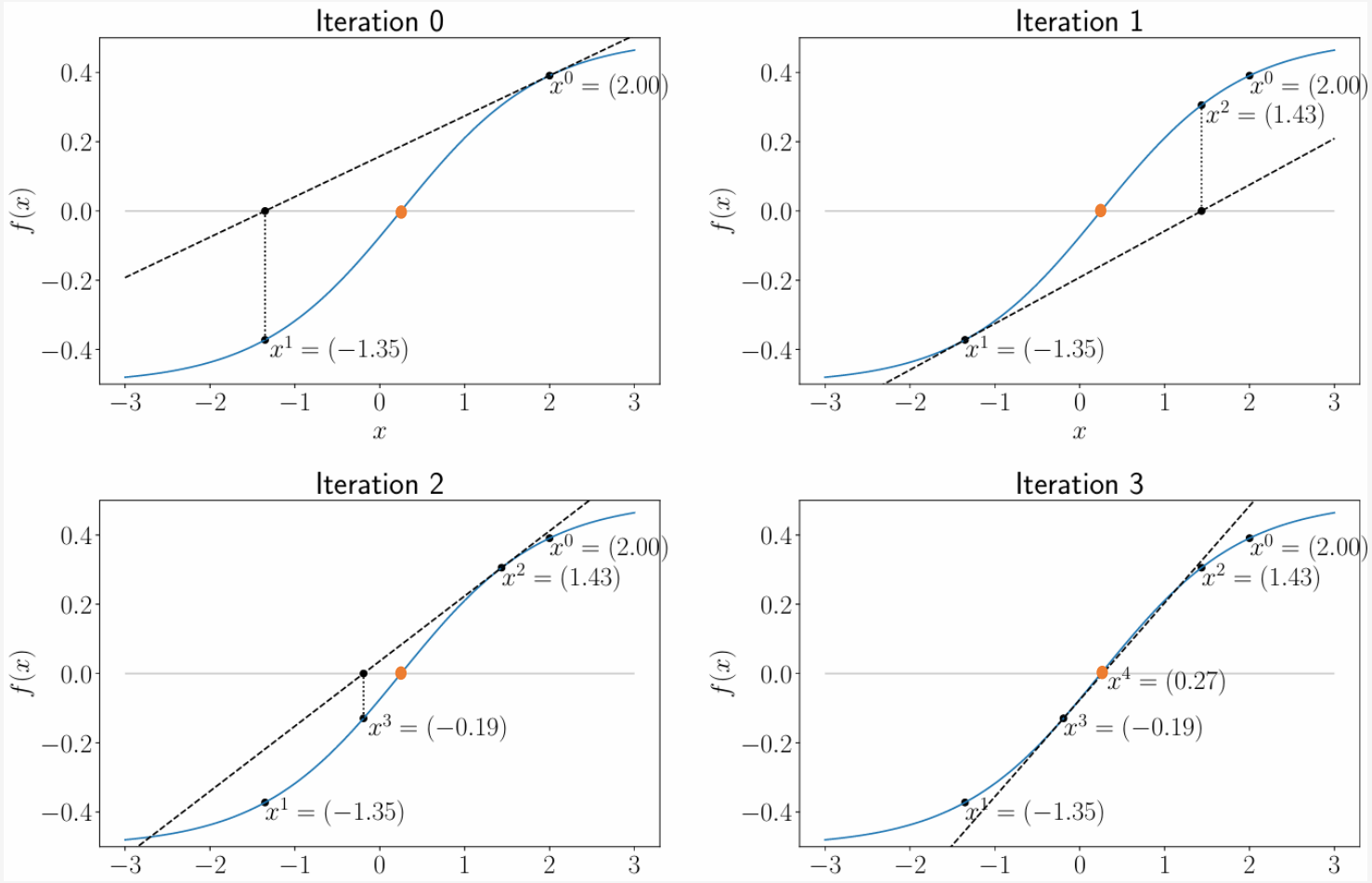
3. Set its solution to $x^{k+1} = \hat{x}$, increase counter $k \leftarrow k + 1$ and **Repeat**

Newton's method – Example

$$f(x) = \frac{1}{1 + \exp(-1.2x + 0.3)} - 0.5$$

$$f(x) = 0$$

$Solution$: $x^* = 0.3$



Newton's root finding method (multivariate functions)

- **Goal**: solve the equation (find the roots of the univariate function h: $R^n \to R^m$):

$$h(x) = 0 \quad \to \quad \begin{matrix} h_1(x) = 0 \\ \vdots \\ h_m(x) = 0 \end{matrix} \quad \to \quad \begin{pmatrix} h_1(x) \\ \vdots \\ h_m(x) \end{pmatrix} = 0, \qquad x \in R^n$$

- The method consists of **3 main iterations** (we start with **initial point $\boldsymbol{x^0}$** at iteration 0)

1. Approximate h by its 1st order Taylor expansion at the current point x^k

$$h(x) \approx \hat{h}(x) = h(x^k) + \nabla^2 h(x^k)(x - x^k)$$

2. Solve the **linear equations**:

$$h(x^k) + \nabla^2 h(x^k)(x - x^k) = 0 \quad \to \quad \hat{x}$$

3. Set its solution to $\boldsymbol{x^{k+1} = \hat{x}}$, $\boldsymbol{k \leftarrow k + 1}$ and **Repeat**

Alternative notation: $\boldsymbol{Dh(x^k) \equiv \nabla^2 h(x^k)}$

Derivative of multivariate function h

$$\nabla^2 h(x^k) = \begin{bmatrix} \frac{\partial h_1(x)}{\partial x_1} & \cdots & \frac{\partial h_1(x)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_m(x)}{\partial x_1} & \cdots & \frac{\partial h_m(x)}{\partial x_n} \end{bmatrix} \in R^{m \times n}$$

Called: **Hessian Matrix**

Newton method iterations

$$h(x^k) + \nabla^2 h(x^k)(x - x^k) = 0$$

Set $\boldsymbol{\Delta x} = x - x^k$

$$h(x^k) + \nabla^2 h(x^k)\boldsymbol{\Delta x} = 0$$

Iterations:

0. Set $k = 0$ and start at a given x^0
1. Solve system of linear equations:
$$\nabla^2 h(x^k)\boldsymbol{\Delta x} = 0$$
2. Set:
$$x^{k+1} = x^k + \boldsymbol{\Delta x}$$
3. Set $k \leftarrow k + 1$
4. Repeat from (1) until $\boldsymbol{\Delta x} \approx \mathbf{0}$

- **Remark #1**: Newton iterations can be **computational expensive** (solution of linear system)
- **Remark #2:** **Fast convergence** (when started close to the solutions x^*)
- **Remark #3:** **May diverge** when started far from x^*

Linear optimization as root finding problem

Optimality conditions

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax \leq b \end{aligned}$$

Primal

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax + s = b \\ & s \geq 0 \end{aligned}$$

Dual

$$\begin{aligned} \min \quad & -b^T x \\ \text{s.t.} \quad & A^T y + c = 0 \\ & y \geq 0 \end{aligned}$$

KKT conditions

$$Ax + s - b = 0$$

$$A^T y + c = 0$$

$$s_i y_i = 0, \;\; i = 1, \ldots, m$$

$$s, y \geq 0$$

Linear optimization as root finding problem

KKT conditions

$$Ax + s - b = 0$$

$$A^T y + c = 0$$

$$s_i y_i = 0, \;\; i = 1, \dots, m$$

$$s, y \geq 0$$

Diagonalize complementarity slackness

$$S = diag(s) = \begin{bmatrix} s_1 & & & \\ & s_2 & & \\ & & \ddots & \\ & & & s_m \end{bmatrix}, \quad Y = diag(y) = \begin{bmatrix} y_1 & & & \\ & y_2 & & \\ & & \ddots & \\ & & & y_m \end{bmatrix}$$

$$SY\mathbf{1} = diag(s)diag(y)\mathbf{1} = \begin{bmatrix} s_1 y_1 & & & \\ & s_2 y_2 & & \\ & & \ddots & \\ & & & s_m y_m \end{bmatrix} \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} = \begin{pmatrix} s_1 y_1 \\ s_2 y_2 \\ \vdots \\ s_m y_m \end{pmatrix}$$

Hence: $s_i y_i = 0, \;\; i = 1, \dots, m \quad \Leftrightarrow \quad SY\mathbf{1}$

Main idea of IPM

Rewrite KKT conditions as:

$$h(y,x,s) = \begin{bmatrix} h_1(y,x,s) \\ h_2(y,x,s) \\ h_3(y,x,s) \end{bmatrix} = \begin{bmatrix} Ax + s - b \\ A^T y + c \\ SY\mathbf{1} \end{bmatrix} = \begin{bmatrix} r_p \\ r_d \\ SY\mathbf{1} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = 0$$

$$s, y \geq 0$$

System of linear and nonlinear equalities and linear inequalities

Main steps:

- Apply **variants of Newton's method** to solve the linear equations: $h(y,x,s) = 0$
- Make sure the $\boldsymbol{s, y > 0}$ (strictly positive) at every iteration
  - **Motivation**: if $s, y > 0$ we **avoid** getting close to the corners of the feasible region

Newton's method for optimality (KKT) conditions

KKT conditions:

$$h(y,x,s)=\begin{pmatrix}h_1(y,x,s)\\h_2(y,x,s)\\h_3(y,x,s)\end{pmatrix}=\begin{pmatrix}Ax+s-b\\A^Ty+c\\SY\mathbf{1}\end{pmatrix}=\begin{pmatrix}r_p\\r_d\\SY\mathbf{1}\end{pmatrix}=0$$

$$s,y\geq 0$$

Derivative of h (Hessian matrix)

$$\nabla^2 h(y,x,s)=\begin{bmatrix}0 & A & I\\A^T & 0 & 0\\S & 0 & Y\end{bmatrix}$$

Iterations:

- Solve: $\nabla^2 h(y^k,x^k,s^k)\Delta(y^k,x^k,s^k)=-h(y^k,x^k,s^k) \quad\Leftrightarrow\quad \begin{bmatrix}0 & A & I\\A^T & 0 & 0\\S^k & 0 & Y^k\end{bmatrix}\begin{pmatrix}\Delta y^k\\\Delta x^k\\\Delta s^k\end{pmatrix}=-\begin{pmatrix}Ax^k+s^k-b\\A^Ty^k+c\\S^kY^k\mathbf{1}\end{pmatrix}$

- Set: $\begin{pmatrix}y^{k+1}\\x^{k+1}\\s^{k+1}\end{pmatrix}=\begin{pmatrix}y^k\\x^k\\s^k\end{pmatrix}+\begin{pmatrix}\Delta y^k\\\Delta x^k\\\Delta s^k\end{pmatrix}$

Caution: it might set $s^k, y^k < 0$.
In that case the system is **not solvable**!

- **Central path**

Line-search to stay strictly feasible $(s, y > 0)$

Root finding equations

$$h(y,x,s) = \begin{bmatrix} h_1(y,x,s) \\ h_2(y,x,s) \\ h_3(y,x,s) \end{bmatrix} = \begin{bmatrix} Ax + s - b \\ A^T y + c \\ SY\mathbf{1} \end{bmatrix} = \begin{bmatrix} r_p \\ r_d \\ SY\mathbf{1} \end{bmatrix} = 0$$

Newton (linear) system:

$$\underbrace{\begin{bmatrix} 0 & A & I \\ A^T & 0 & 0 \\ S^k & 0 & Y^k \end{bmatrix}}_{\nabla^2 h^k} \begin{pmatrix} \Delta y^k \\ \Delta x^k \\ \Delta s^k \end{pmatrix} = -\underbrace{\begin{pmatrix} r_p^k \\ r_d^k \\ S^k Y^k \mathbf{1} \end{pmatrix}}_{h^k}$$

Residuals:

$$r_p^k = Ax^k + s^k - b$$
$$r_d^k = A^T y^k + c$$

Line-search to enforce: $s, y > 0$

$$\begin{pmatrix} y^{k+1} \\ x^{k+1} \\ s^{k+1} \end{pmatrix} = \begin{pmatrix} y^k \\ x^k \\ s^k \end{pmatrix} + \alpha \begin{pmatrix} \Delta y^k \\ \Delta x^k \\ \Delta s^k \end{pmatrix}$$

Computational Issue:

- Pure Newton step does not make significant progress towards a solution:

$$h(y,x,s) = 0, \qquad s, y \geq 0$$

Smoothed optimality conditions & The central path

Perturbed optimality conditions

$$Ax + s - b = 0$$

$$A^T y + c = 0$$

perturbation

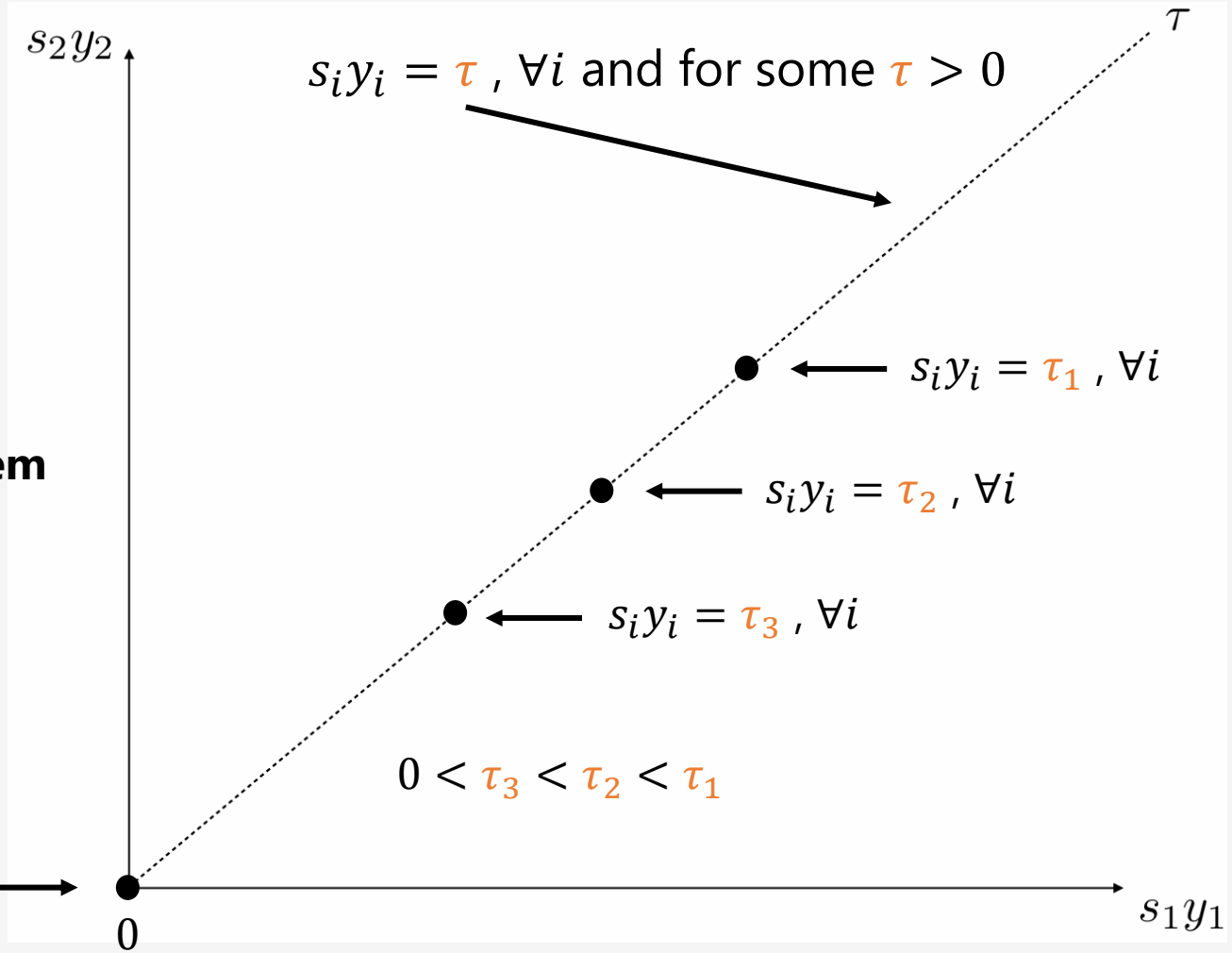
$$s_i y_i = \tau, \quad i = 1, \dots, m$$

$$s, y \geq 0$$

Same optimality conditions for a **smoothed problem**

Duality gap:

$$b^T y + c^T x = b^T y - x^T A^T y = (b - Ax)^T y = s^T y$$



Newton's method for smoothed optimality conditions

Smoothed optimality conditions

$$h(y,x,s) = \begin{pmatrix} h_1(y,x,s) \\ h_2(y,x,s) \\ h_3(y,x,s) \end{pmatrix} = \begin{pmatrix} Ax + s - b \\ A^T y + c \\ SY\mathbf{1} - \tau\mathbf{1} \end{pmatrix} = 0$$

Newton (linear) system:

$$\begin{bmatrix} 0 & A & I \\ A^T & 0 & 0 \\ S^k & 0 & Y^k \end{bmatrix} \begin{pmatrix} \Delta y^k \\ \Delta x^k \\ \Delta s^k \end{pmatrix} = - \begin{pmatrix} r_p^k \\ r_d^k \\ S^k Y^k \mathbf{1} - \tau\mathbf{1} \end{pmatrix} = \begin{pmatrix} -r_p^k \\ -r_d^k \\ -S^k Y^k \mathbf{1} + \tau\mathbf{1} \end{pmatrix}$$

Line-search to enforce: $s, y > 0$

$$\begin{pmatrix} y^{k+1} \\ x^{k+1} \\ s^{k+1} \end{pmatrix} = \begin{pmatrix} y^k \\ x^k \\ s^k \end{pmatrix} + \alpha \begin{pmatrix} \Delta y^k \\ \Delta x^k \\ \Delta s^k \end{pmatrix}$$

The path parameter τ

We set as **duality measure**

$$\mu = \frac{s^T y}{m}$$ (the **average** value of the pairs $s_i y_i$)

And set the perturbation: $\tau = \sigma\mu$

Newton (linear) system:

$$\begin{bmatrix} 0 & A & I \\ A^T & 0 & 0 \\ S^k & 0 & Y^k \end{bmatrix} \begin{pmatrix} \Delta y^k \\ \Delta x^k \\ \Delta s^k \end{pmatrix} = \begin{pmatrix} -r_p^k \\ -r_d^k \\ -S^k Y^k \mathbf{1} + \sigma\mu \mathbf{1} \end{pmatrix}$$

Centering parameter: $\sigma \in [0,1]$

For $\sigma = 0$ → Pure Newton step

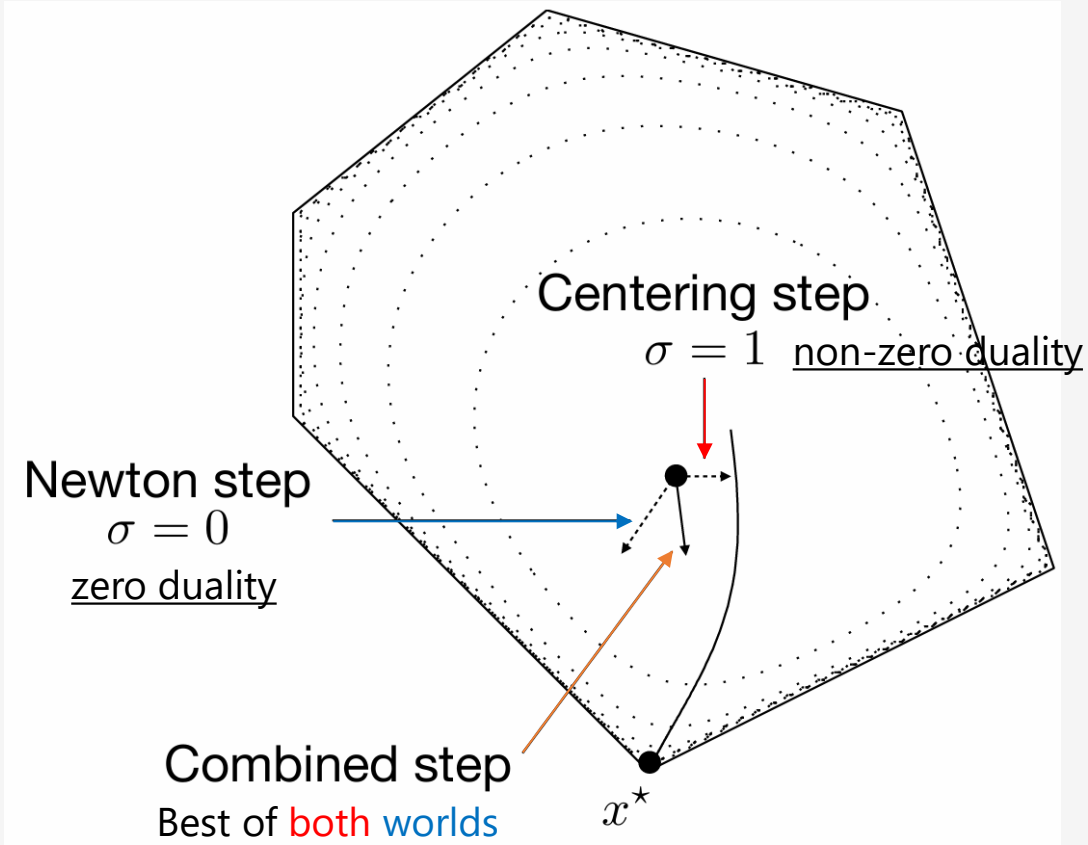
For $\sigma = 1$ → Centering step towards $\left(y^*(\mu), x^*(\mu), s^*(\mu)\right)$

Line-search to enforce: $s, y > 0$

$$\begin{pmatrix} y^{k+1} \\ x^{k+1} \\ s^{k+1} \end{pmatrix} = \begin{pmatrix} y^k \\ x^k \\ s^k \end{pmatrix} + \alpha \begin{pmatrix} \Delta y^k \\ \Delta x^k \\ \Delta s^k \end{pmatrix}$$

- **Primal-dual path-following algorithm**

Path-following idea



Centering step:

It brings towards the central path and is usually biased towards $s, y > 0$. No progress on duality measure, since $\mu \neq 0$ or $\sigma \neq 0$.

Newton step:

It brings the iterates towards the zero duality ($\mu = \sigma = 0$). It quickly violates $s, y > 0$.

Combined step:

Best of both worlds, with longer steps

Primal-dual path-following algorithm

Initialization:

Given starting point (y^0, x^0, s^0) with $y^0, s^0 > 0$. Set also $k = 0$

Iterations:

1. Choose $\sigma \in [0,1]$

Perturbed Newton system

2. Solve linear system: $\begin{bmatrix} 0 & A & I \\ A^T & 0 & 0 \\ S^k & 0 & Y^k \end{bmatrix} \begin{pmatrix} \Delta y^k \\ \Delta x^k \\ \Delta s^k \end{pmatrix} = \begin{pmatrix} -r_p^k \\ -r_d^k \\ -S^k Y^k \mathbf{1} + \boldsymbol{\sigma\mu}\mathbf{1} \end{pmatrix}$, where $\mu = \left(s^k\right)^T y^k / m$

3. Find maximum step size α such that $y^k + \alpha \Delta y^k > 0$ and $s^k + \alpha \Delta s^k > 0$

4. Compute next iterate $\begin{pmatrix} y^{k+1} \\ x^{k+1} \\ s^{k+1} \end{pmatrix} = \begin{pmatrix} y^k \\ x^k \\ s^k \end{pmatrix} + \alpha \begin{pmatrix} \Delta y^k \\ \Delta x^k \\ \Delta s^k \end{pmatrix}$, set $k = k + 1$ and GoTo Step 1.

Working towards ***optimality conditions***

Optimality conditions satisfied only at convergence (i.e., as $\sigma \to 0$)

Primal residual:

$$r_p = Ax + s - b \to 0$$

Stopping criteria:

$$\left|\left|r_p\right|\right| \leq \epsilon_{primal}$$

Dual residual:

$$r_d = A^t y + c \to 0$$

$$\left|\left|r_d\right|\right| \leq \epsilon_{dual}$$

Complementarity slackness:

$$s^T y \to 0$$

$$s^T y \leq \epsilon_{gap}$$

- **Logarithmic barrier functions**

Smoothed optimality conditions

Perturbed optimality conditions

$$Ax + s - b = 0$$

$$A^T y + c = 0$$

$$s_i y_i = \tau, \;\; i = 1, \ldots, m$$

$$s, y \geq 0$$

Same optimality conditions for a smoothed problem

Duality gap:

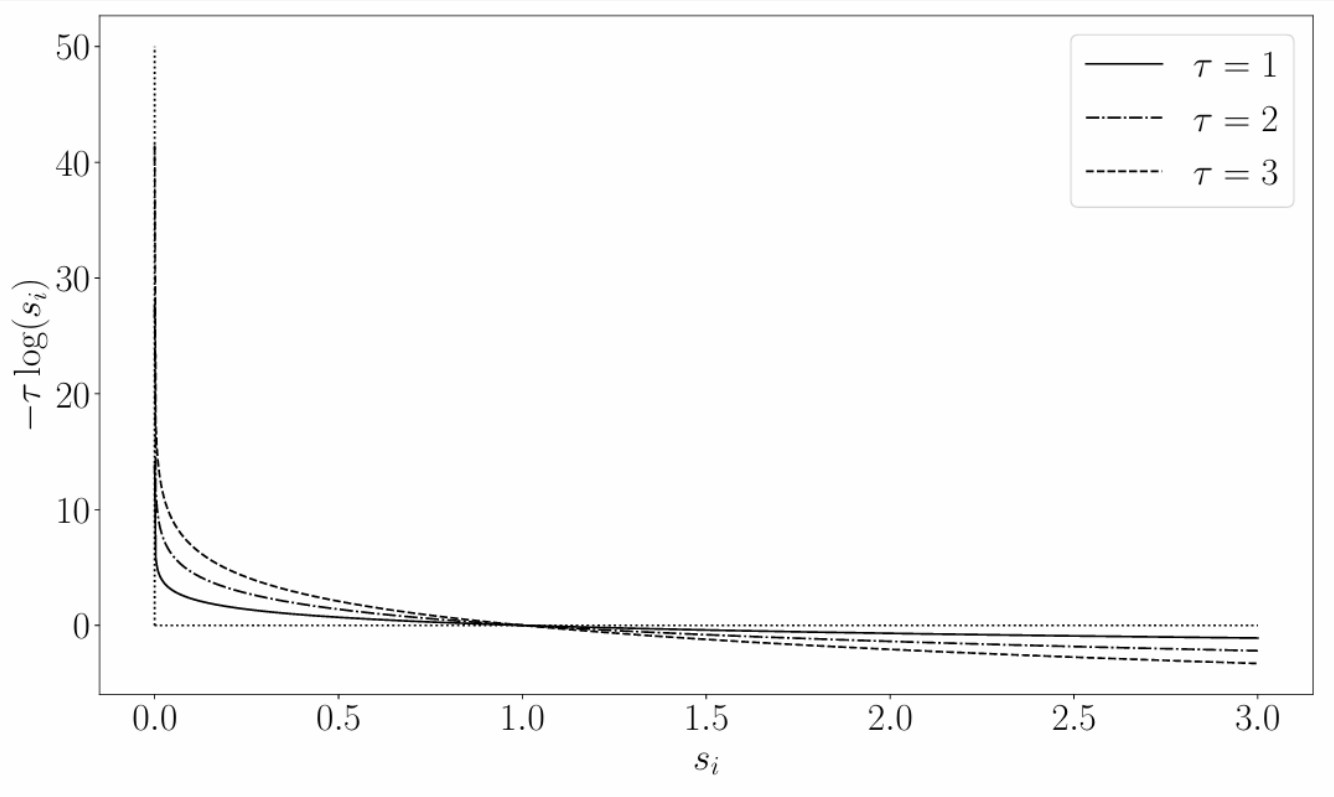
$$b^T y + c^T x = b^T y - x^T A^T y = (b - Ax)^T y = s^T y$$

Do solutions actually exist?

What do they represent?

Logarithmic barrier function

$\phi: (0, \infty) \rightarrow R$ with $\phi(s) = -\tau \sum_{i=1}^{m} \log(s_i)$



As $\tau \rightarrow 0$ it approximates

$$I_{s_i \geq 0} = \begin{cases} 0, & if \;\; s_i \geq 0 \\ \infty, & otherwise \end{cases}$$

Smoothed problem

$$\begin{aligned} \min \;\; & c^T x \\ \text{s.t.} \;\; & Ax + s = b \\ & s \geq 0 \end{aligned} \quad \Rightarrow \quad \begin{aligned} \min \;\; & c^T x + \phi(s) = c^T x \; - \tau \sum_{i=1}^{m} \log(s_i) \\ \text{s.t.} \;\; & Ax + s = b \end{aligned}$$

Barrier problem

Lagrangian function

$$L(x, s, y) = \; c^T x \; - \tau \sum_{i=1}^{m} \log(s_i) + y^T (Ax + s - b)$$

Derivatives of Lagrangian wrt x, s

$$\frac{\partial L}{\partial x} = c + A^T y = 0$$

$$\frac{\partial L}{\partial s_i} = \tau \frac{1}{s_i} + y_i = 0 \;\Rightarrow\; s_i y_i = \tau$$

Central path

$\tau \approx +\infty \rightarrow$ analytic center

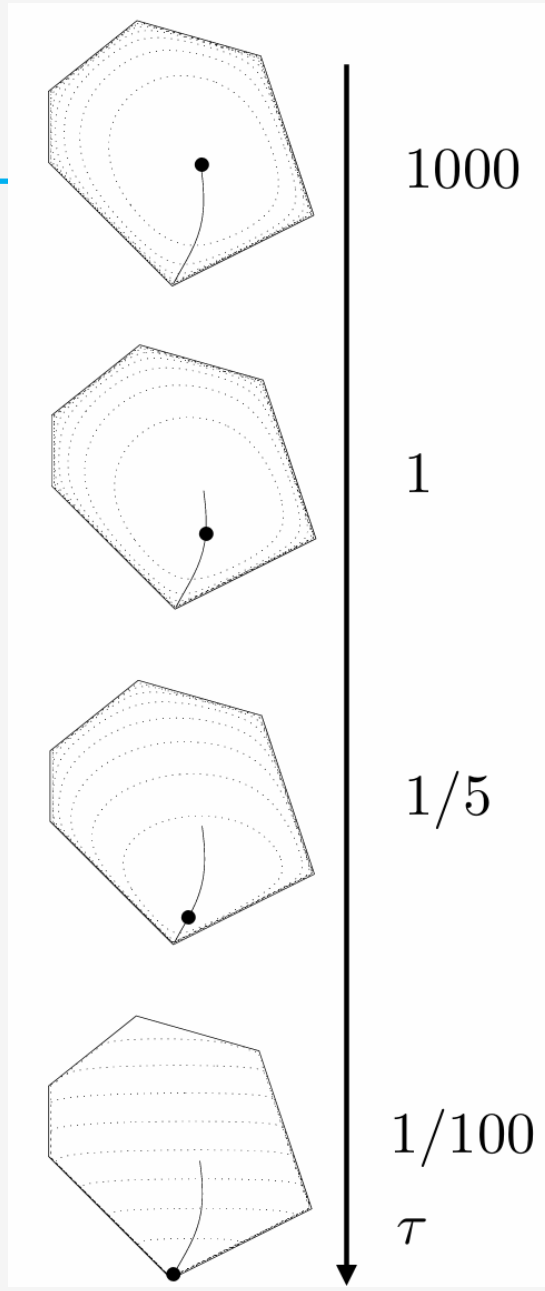
Barrier problem

$$\min \; c^T x + \phi(s) = c^T x \; - \tau \sum_{i=1}^{m} \log(s_i)$$
$$\text{s.t. } Ax + s = b$$

For a decreasing sequence of $\tau > 0$ we obtain the set of points: $\left(x^*(\tau), s^*(\tau), y^*(\tau)\right)$ that are **solutions of the linear system (perturbed optimality conditions)**:

$$Ax + s - b = 0$$
$$A^T y + c = 0$$
$$s_i y_i = \tau, \;\; i = 1, \dots, m$$
$$s, y \geq 0$$

Main idea:
follow the central path as $\tau \to 0$



references

- B/T:
  - Ch. 9.4-9.6: Interior Point Methods
- V:
  - Ch. 17: The central path
  - Ch. 15: A path-following method

Next lecture

- Practical interior-point method (Mehrotra predictor-corrector algorithm)
- Implementation details
- Interior Point Algorithm VS Simplex method